

ITA0606 – Machine Learning

Exp-6

Program:

Program to implement Naïve Bayes Algorithm

and display Confusion Matrix with Accuracy

Import required libraries

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import confusion_matrix, accuracy_score

Step 1: Load the Iris dataset

iris = load_iris()

X = iris.data # Input features

y = iris.target # Target labels

Step 2: Split the dataset into Training and Testing sets

X_train, X_test, y_train, y_test = train_test_split(

 X, y,

 test_size=0.3,

 random_state=42

)

Step 3: Create the Naïve Bayes model

model = GaussianNB()

Step 4: Train the model

```
model.fit(X_train, y_train)
```

```
# Step 5: Predict the test data
```

```
y_pred = model.predict(X_test)
```

```
# Step 6: Display Actual and Predicted Labels
```

```
print("=" * 60)
```

```
print("Actual Labels:")
```

```
print(y_test)
```

```
print("\nPredicted Labels:")
```

```
print(y_pred)
```

```
# Step 7: Generate Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\n" + "=" * 60)
```

```
print("Confusion Matrix")
```

```
print("=" * 60)
```

```
print(cm)
```

```
# Step 8: Detailed Explanation of the Confusion Matrix
```

```
class_names = iris.target_names
```

```
print("\nDetailed Interpretation of Confusion Matrix")
```

```
print("-" * 60)
```

```
for i in range(len(class_names)):
```

```

print(f"\nActual Class : {class_names[i]}")

for j in range(len(class_names)):

    print(f"Predicted as {class_names[j]:12s} : {cm[i][j]} sample(s)")


# Step 9: Overall Interpretation

correct_predictions = cm.trace()

total_predictions = cm.sum()

incorrect_predictions = total_predictions - correct_predictions


print("\n" + "=" * 60)

print("Summary")

print("=" * 60)

print(f"Total Test Samples      : {total_predictions}")

print(f"Correct Predictions       : {correct_predictions}")

print(f"Incorrect Predictions      : {incorrect_predictions}")


print("\nCorrectly Classified Samples (Diagonal Elements):")

for i in range(len(class_names)):

    print(f"{class_names[i]:12s} -> {cm[i][i]} sample(s)")


print("\nMisclassified Samples (Off-Diagonal Elements):")

misclassified = False

for i in range(len(class_names)):

    for j in range(len(class_names)):

        if i != j and cm[i][j] > 0:

            print(f"{cm[i][j]} sample(s): Actual '{class_names[i]}' predicted as '{class_names[j]}'")

            misclassified = True

```

if not misclassified:

```
print("No misclassified samples.")
```

Step 10: Calculate Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\n" + "=" * 60)
```

```
print(f"Accuracy of the Model : {accuracy * 100:.2f}%")
```

```
print("=" * 60)
```

OUTPUT:

```
=====
Actual Labels:
[1 0 2 1 1 0 1 2 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Predicted Labels:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 2 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

=====
Confusion Matrix
=====
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]

Detailed Interpretation of Confusion Matrix
-----

Actual Class : setosa
Predicted as setosa      : 19 sample(s)
Predicted as versicolor : 0 sample(s)
Predicted as virginica  : 0 sample(s)

Actual Class : versicolor
Predicted as setosa      : 0 sample(s)
Predicted as versicolor : 12 sample(s)
Predicted as virginica  : 1 sample(s)

Actual Class : virginica
Predicted as setosa      : 0 sample(s)
Predicted as versicolor : 0 sample(s)
Predicted as virginica  : 13 sample(s)

=====
Summary
=====
Total Test Samples      : 45
Correct Predictions     : 44
Incorrect Predictions   : 1

Correctly Classified Samples (Diagonal Elements):
setosa      -> 19 sample(s)
versicolor  -> 12 sample(s)
virginica   -> 13 sample(s)

Misclassified Samples (Off-Diagonal Elements):
1 sample(s): Actual 'versicolor' predicted as 'virginica'

=====
Accuracy of the Model : 97.78%
=====
```